Table of Contents

Instructor Lab Guide: Automating VLAN Configuration Using Python and NETCONF ... 2

Instructor Note 2

Objectives 2

Background / Scenario 2

Overview 2

Learning Outcomes 3

Lab Environment 3

Instructor Preparation Checklist 3

Instructor Walkthrough 3

Required Router Configuration (Solution Setup) 4

Expected Configuration Output 4

Expected Student Outputs 4

Instructor Notes 6

Optional Enhancements 7

Reflection / Discussion Questions 7

Instructor Lab Guide: Automating VLAN Configuration Using Python and NETCONF

Instructor Note

This guide contains expected outputs, solutions, and troubleshooting guidance intended for instructor use only.

Objectives

Part 1: Configure router for NETCONF and SSH access

Part 2: Execute Python script to create VLAN subinterface

Part 3: Verify configuration using NETCONF

Part 4: Validate configuration using CLI commands

Part 5: Troubleshoot connectivity and authentication issues

Background / Scenario

Today's network automation tools today leverage model-driven interfaces like NETCONF rather than automation over the CLI. NETCONF allows structured, reliable access to network devices instead of having to parse unstructured output.

The goal of this lab is to configure a VLAN subinterface on a Cisco CSR1000v router using Python code. The student will perform this configuration using the `ncclient` library.. The student will accomplish this task using the `ncclient` library. We will verify the configuration by issuing NETCONF commands as well as traditional CLI commands. Students should understand that even though we are automating configurations we should always validate our configurations.

Overview

Students will learn how to automate the configuration of a network device by using Python and NETCONF on a Cisco CSR1000v router. The students will use Python scripts to configure a VLAN subinterface. The students will validate if they have successfully configured their device by using NETCONF as well as CLI commands. The lab's focus is on automation as well as validation.

Learning Outcomes

Students should be able to:

- Establish a NETCONF session using Python
- Configure a VLAN subinterface using automation
- Understand 802.1Q encapsulation
- Verify configuration using both API and CLI
- Troubleshoot authentication and connectivity issues

Lab Environment

This lab can be completed in:

- NDG NetLab DevNet environment
- Local DevNet VM with CSR1000v router
- NETCONF operates over TCP port 830, which must be reachable from the DevNet VM.

Note: NetLab environments may reset between sessions, requiring reconfiguration.

Instructor Preparation Checklist

Before students begin, verify:

- Router is accessible
- NETCONF is enabled
- SSH is configured
- Local user account exists
- Python and `ncclient` are installed
- Students can ping router

Instructor Walkthrough

Step 1: Verify environment readiness
Ensure students can access the router and that NETCONF and SSH are enabled.

Step 2: Validate script execution
Students should successfully run `vlan_config.py` and observe the success message.

Step 3: Confirm NETCONF output
Students should run `verify_vlan.py` and review XML output for correct configuration.

Step 4: Perform CLI validation
Students should verify the configuration using show commands.

Step 5: Troubleshoot issues
Guide students through authentication, connectivity, and configuration errors as needed.

Required Router Configuration (Solution Setup)

```
conf t
username cisco privilege 15 password cisco
ip domain-name lab.local
crypto key generate rsa modulus 1024
line vty 0 4
login local
transport input ssh
netconf-yang
end
write memory
```

Expected Configuration Output

After running `vlan_config.py`, the router should contain:

```
interface GigabitEthernet1.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0
description VLAN10_SUBINTERFACE
```

Expected Student Outputs

The following outputs indicate successful completion of the lab:

1) Script output: After running the `vlan_config.py` script, students should see the following output, as shown in Figure 1:

```
GigabitEthernet1.10 successfully configured for VLAN 10
```

```
devasc@labvm: ~
File  Edit  View  Search  Terminal  Help
devasc@labvm:~$ python3 vlan_config.py
GigabitEthernet1.10 successfully configured for VLAN 10
devasc@labvm:~$
```

Figure 1. Successful Execution of vlan_config.py Script

As shown in Figure 1, the Python script executes successfully and confirms that the VLAN subinterface configuration was applied to the router using NETCONF.

2) NETCONF Output: Students should see XML including:

- Interface name: 1.10
- VLAN ID: 10
- IP address: 192.168.10.1
- Subnet mask: 255.255.255.0
- Description: VLAN10_SUBINTERFACE

```
devasc@labvm:~$ python3 verify_vlan.py
<?xml version="1.0" encoding="UTF-8"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:d7dbb3ba-1bc0-4d34-84ca-2f786b3290b0" xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0"><data><native xmlns="http://cisco.com/ns/y
ang/Cisco-IOS-XE-native"><interface><GigabitEthernet><name>1</name><ip><address><primary><address>192.168.56.101</address><mask>255.255.255.0</mask></primary></address></ip><mop><enabled>false</enabled><sysid>fa
lse</sysid></mop><negotiation xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-ethernet"><auto>true</auto></negotiation></GigabitEthernet><GigabitEthernet><name>1.10</name><description>VLAN10_SUBINTERFACE</descripti
on><encapsulation><dot1Q><vlan-id>10</vlan-id></dot1Q></encapsulation><ip><address><primary><address>192.168.10.1</address><mask>255.255.255.0</mask></primary></address></ip></GigabitEthernet></interface></nativ
e></data></rpc-reply>
devasc@labvm:~$
```

Figure 2. NETCONF XML Output Showing VLAN Subinterface Configuration

As shown in Figure 2, the NETCONF output confirms that the subinterface GigabitEthernet1.10 has been successfully configured with VLAN ID 10, the assigned IP address, and the appropriate description.

3) CLI Verification (Operational Validation)

A. `show ip interface brief`

Should show: `GigabitEthernet1.10 192.168.10.1` as shown in Figure 3:

```
CSR1kv#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet1       192.168.56.101  YES NVRAM  up                    up
GigabitEthernet1.10    192.168.10.1    YES other  up                    up
CSR1kv#
```

Figure 3. Interface Summary Showing Configured Subinterface

B. `show running-config interface GigabitEthernet1.10`

Should include: `encapsulation dot1Q 10`

`ip address 192.168.10.1 255.255.255.0`

`description VLAN10_SUBINTERFACE`

```
CSR1kv#show running-config interface GigabitEthernet1.10
Building configuration...

Current configuration : 133 bytes
!
interface GigabitEthernet1.10
 description VLAN10_SUBINTERFACE
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
end
```

Figure 4. Running Configuration of GigabitEthernet1.10.

Common Student Errors

Issue	Cause	Solution
Authentication failed	SSH/local login not configured	Configure username `+ login local`
Cannot connect	Wrong IP	Verify with `show ip interface brief`
Script fails	NETCONF not enabled	Enable `netconf-yang`
SSH fails	Missing RSA keys	Generate keys
Works locally but not in NetLab	Environment reset	Reconfigure router

Instructor Notes

- Students often confuse router IP vs VLAN IP
- Emphasize:
  - 192.168.56.x → router management (connection) IP
  - 192.168.10.1 → VLAN subinterface IP
- NETCONF uses port 830
- Subinterfaces are used instead of VLAN database on routers

This lab reinforces the importance of combining automation with validation to ensure reliable and repeatable network configurations.

Optional Enhancements

Students may:

- Change VLAN ID
- Add additional subinterfaces
- Expand verification script
- Automate multiple configurations

Reflection / Discussion Questions

1. What benefits does NETCONF offer compared to typical CLI configuration?

   NETCONF allows devices to be accessed using XML data models, eliminating much of the parsing normally required when working with unstructured CLI outputs. NETCONF can also use transactions when pushing configuration changes to the network. Instead of issuing CLI commands directly, NETCONF provides many opportunities for automation, consistency, idempotency, and scalability.

2. How can consistency in network management be improved with automation?

   Automation provides consistency. Configurations applied to devices are the same each time with no variation from manual entry. Automation enables scripts/templates that can be defined once and then run many times, which decreases the chance for human error. This allows for standardized configurations to be rolled out across your network.

3. What challenges did you experience during this lab?

   Problems can occur due to authentication issues, incorrect IP address, or NETCONF/SSH enablement on the router. Also, when working in NetLab, students may experience problems where they lose their work between sessions or the session gets reset. Troubleshooting should be approached as if these issues were occurring on a local router.

4. Why should you validate after applying configurations?

   Validation ensures your configurations are applied successfully and functioning properly. Without validating configurations, misconfigurations may go unnoticed. These oversights can lead to network instability or downtime. Validate your configurations using NETCONF and CLI commands to ensure proper configuration and identify problems sooner.